

Question Code: Q331

- Open the Learnlytica assessment tool
- Enter your Email address, and then enter the Question ID and click on start assessment
- Once started, click on VSCode, and it will directly open the solution.py file of the question, and write your code there.
- Once you have written the code, come back to the tool and click on run tests. After executing the run tests, click on Validate.
- After completing that validation, and you are satisfied with your attempt, you can click on submit in the tool and then click End assessment.

■ Problem Statement

You need to implement a **Customer Sentiment Analyzer** that takes a customer review as input and analyzes the sentiment-related words. It should return a dictionary where:

- Keys → Sentiment words
- Values → Sentiment word frequencies

The implementation should handle punctuation and case insensitivity.

■ Class Declaration

class CustomerSentimentAnalyzer:

■ No need for init method

■ If used, follow recommendations -

```
def __init__(self):  
    pass
```

■ Operations

1. Clean and Tokenize Customer Review

■ Removes punctuation, converts the review to lowercase, and tokenizes words.

■ Function Prototype: **def preprocess_text(self, text: str) -> list:**

■ Example Input: **preprocess_text("Excellent service, excellent support! Very helpful.")**

■ Expected Output: **['excellent', 'service', 'excellent', 'support', 'very', 'helpful']**

■ Implementation Flow:

■ Convert the input text to lowercase.

■ Remove punctuation using **re.sub(r'^\wls]', "", text)**.

■ Split text into words using **.split()**.

2. Compute Sentiment Word Frequency

■ Counts occurrences of each word in a dictionary.

■ Function Prototype: **def compute_word_frequency(self, words: list) -> dict:**

■ Example Input: **compute_word_frequency(['excellent', 'service', 'excellent', 'support', 'very', 'helpful'])**

■ Expected Output: **{'excellent': 2, 'service': 1, 'support': 1, 'very': 1, 'helpful': 1}**

■ Implementation Flow:

■ Declare empty dictionary.

■ Iterate through the list using for loop and use **dictionary.get(word, 0)** to count occurrences.

■ Return a dictionary where keys = words and values = frequencies.

3. Get Most Frequent Sentiment Word

■ Returns the most frequent sentiment word and its count.

■ Function Prototype: **def get_most_frequent_word(self, freq_dict: dict) -> tuple:**

■ Example Input: **get_most_frequent_word({'excellent': 2, 'service': 1, 'support': 1, 'helpful': 1})**

■ Expected Output: **('excellent', 2)**

- Implementation Flow:
- If nothing in dictionary, return None.
- Iterate through the dictionary to find the word with the highest count. Use **max** for highest.
- Use **lambda x:x[1]** for mapping keys.
- Return the word and its count as a tuple.

4. Filter Sentiment Words by Minimum Frequency

- Returns words that appear at least n times.
- Function Prototype: **def filter_words_by_frequency(self, freq_dict: dict, n: int) -> dict:**
- Example Input: **filter_words_by_frequency({'excellent': 3, 'service': 2, 'support': 1, 'helpful': 1}, 2)**
- Expected Output: **{'excellent': 3, 'service': 2}**
- Implementation Flow:
- If count > n, iterate over the dictionary using for loop and include only words that meet or exceed the threshold & return a filtered dictionary.
- Else return empty dictionary.

Question Code: Q330

- Open the Learnlytica assessment tool
- Enter your Email address, and then enter the Question ID and click on start assessment
- Once started, click on VSCode, and it will directly open the solution.py file of the question, and write your code there.
- Once you have written the code, come back to the tool and click on run tests. After executing the run tests, click on Validate.
- After completing that validation, and you are satisfied with your attempt, you can click on submit in the tool and then click End assessment.

■ Problem Statement

A library tracks daily book lending logs. Each log entry includes the member ID, book category, date, and lending status (Borrowed/Returned/Overdue). Your task is to perform analysis over this lending data to identify return rates, overdue patterns, and category-wise summaries.

■ Declare Class

class LibraryLendingAnalyzer:

No need to use init method

If needed, follow recommendations:

```
def __init__(self):  
pass
```

■ Operations

■ 1. Create Lending DataFrame

Converts raw lending logs into a structured Pandas DataFrame.

Function Prototype: **def create_lending_df(self, data: list) -> pd.DataFrame:**

Example Input:

```
create_lending_df([  
    [501, "Fiction", "2024-06-01", "Borrowed"],  
    [502, "Science", "2024-06-01", "Returned"],  
    [501, "Fiction", "2024-06-02", "Overdue"]  
])
```

Expected Output:

```
MemberID Category Date Status  
0 501 Fiction 2024-06-01 Borrowed  
1 502 Science 2024-06-01 Returned  
2 501 Fiction 2024-06-02 Overdue
```

Implementation Flow:

- Create a DataFrame using columns: ["MemberID", "Category", "Date", "Status"].
- Ensure Date remains as a string (do not convert to datetime).

■ 2. Calculate Monthly Return Rate per Member

Returns a DataFrame showing percentage of lending records marked Returned per member per month.

Function Prototype: **def compute_monthly_return_rate(self, df: pd.DataFrame) -> pd.DataFrame:**

Example Input: (Assuming records from June 2024)

Expected Output:

```
MemberID Month Return Rate  
0 501 2024-06 50.0  
1 502 2024-06 100.0
```

Implementation Flow:

- Extract "Year-Month" from Date using **df["Date"].str[:7]**. This will remove day and keep rest intact.
- Group by MemberID and Month, count and total lending records.
- Create a new df by applying filter over Status = Returned.

- Merge original and Returned df.
- Compute Return Rate = (Returned / Total) * 100.

■ 3. Add Overdue Flag Column

Adds a column "IsOverdue" with 1 if status is "Overdue", else 0.

Function Prototype: `def add_overdue_flag(self, df: pd.DataFrame) -> pd.DataFrame:`

Example Input:

`add_overdue_flag(df)`

Expected Output:

MemberID	Category	Date	Status	IsOverdue	
0	501	Fiction	2024-06-01	Borrowed	0
1	502	Science	2024-06-01	Returned	0
2	501	Fiction	2024-06-02	Overdue	1

Implementation Flow:

- Use `df["Status"] == "Overdue"` and convert boolean to int.

■ 4. Filter Members with High Overdue Records

Returns a DataFrame of members with more than n overdue records.

Function Prototype: `def high_overdue_members(self, df: pd.DataFrame, threshold: int) -> pd.DataFrame:`

Example Input:

`high_overdue_members(df, 1)`

Expected Output:

MemberID	Overdue Count
0	502 3
1	503 4

Implementation Flow:

- Filter rows where Status is "Overdue".
- Group by MemberID and count overdue records.
- Return only those with count > threshold.

■ 5. Category-wise Lending Summary

Returns counts of each lending status per book category.

Function Prototype: `def category_lending_summary(self, df: pd.DataFrame) -> pd.DataFrame:`

Example Input:

`category_lending_summary(df)`

Expected Output:

Category	Borrowed	Returned	Overdue	
0	Fiction	3	2	1
1	Science	2	3	0

Implementation Flow:

- Implement Crosstab using "Category" and "Status" in the input dataframe.
- Set column name to None.
- Use `pd.reindex` with column argument as [Borrowed, Returned, Overdue]. Replace null values with 0; use `fill_value=0`.
- `reset_index` and return the resultant dataframe.